



PUC Minas

Arquitetura de Computadores III

Hardware Multithreading

Motivação para Hardware Multithreading

- Processadores superescalares frequentemente possuem unidades funcionais ociosas
- Dependências de dados e stalls limitam o aproveitamento do hardware
- Longas latências de memória podem deixar o pipeline parado
- Hardware multithreading busca aumentar a utilização dos recursos internos
 - Objetivo principal: melhorar throughput do processador
 - Multithreading tenta manter o hardware ocupado mesmo quando uma thread está parada.

Hardware Multithreading

Relação com MIMD

- MIMD utiliza múltiplos processadores executando múltiplas threads/processos
- Hardware multithreading atua dentro de um único processador
- Várias threads compartilham as mesmas unidades funcionais
- Execução ocorre de forma sobreposta (overlapping execution)
- Explora paralelismo em nível de thread (TLP)

Hardware Multithreading

Recursos Necessários para Hardware Multithreading

- Cada thread precisa possuir estado arquitetural independente
- O processador duplica:
 - Program Counter (PC)
 - Banco de registradores
 - Estado de controle
- A memória principal continua compartilhada
- Memória virtual já fornece mecanismos para multiprogramação
- Apenas o estado da thread é duplicado; as unidades funcionais continuam compartilhadas.

Hardware Multithreading

Troca de Contexto entre Threads

- Hardware deve alternar rapidamente entre threads
- Troca de Thread é muito mais rápida que troca de processo
- Troca de processo pode consumir centenas ou milhares de ciclos
- Troca de thread em hardware pode ocorrer praticamente instantaneamente
- Redução da sobrecarga aumenta o aproveitamento do pipeline

Hardware Multithreading

Classificação das Técnicas de Hardware Multithreading

- Existem três principais abordagens:
 - **Fine-Grained Multithreading**
 - **Coarse-Grained Multithreading**
 - **Simultaneous Multithreading (SMT)**
- Cada técnica faz trade-offs entre:
 - Complexidade
 - Throughput
 - Latência individual
 - Utilização de recursos

Hardware Multithreading

Fine-Grained Multithreading

- Alterna entre threads a cada instrução
- Execução das threads é intercalada
- Escalonamento geralmente ocorre em round-robin
- Threads bloqueadas são ignoradas temporariamente
- Requer capacidade de troca de thread a cada ciclo
- O processador nunca permanece preso a uma única thread por muitos ciclos.

Hardware Multithreading

Fine-Grained Multithreading

- **Vantagens**

- Esconde parada curtas e longas
- Reduz períodos de ociosidade do pipeline
- Mantém unidades funcionais ocupadas
- Melhora a vazão global do processador

- **Desvantagens**

- Cada thread individual executa mais lentamente
- Threads competem continuamente pelos recursos
- Mesmo uma thread sem paradas sofre atrasos
- Pode aumentar a latência individual das aplicações

Hardware Multithreading

Coarse-Grained Multithreading

- Alternância entre threads ocorre apenas em paradas longas
- Exemplo típico:
 - Miss na cache de último nível (LLC miss)
- Durante execução normal, apenas uma thread utiliza o pipeline
- Reduz necessidade de troca extremamente rápida

Hardware Multithreading

Objetivo do Coarse-Grained Multithreading

- Minimizar impacto das paradas de alta latência
- Evitar penalizar threads sem paradas
- Permitir melhor desempenho individual
- Diminuir complexidade de hardware em relação ao fine-grained

Hardware Multithreading

Problema do Coarse-Grained Multithreading

- Pipeline precisa ser congelado ou esvaziado
- Nova thread precisa preencher o pipeline novamente
- Existe sobrecarga de preenchimento do pipeline
- Pequenas paradas não justificam troca de thread
- Eficiência depende da duração das paradas
- O custo de preenchimento do pipeline limita a eficácia contra paradas curtas.

Hardware Multithreading

Comparação: Fine-Grained vs Coarse-Grained

Característica	Fine-Grained	Coarse-Grained
Troca de thread	A cada ciclo	Apenas stalls longos
Overhead de troca	Muito baixo	Moderado
Latência individual	Maior	Menor
Ocultação de stalls	Excelente	Parcial
Complexidade	Alta	Menor

Hardware Multithreading

Simultaneous Multithreading (SMT)

- SMT combina:
 - Paralelismo em nível de thread (TLP)
 - Paralelismo em nível de instrução (ILP)
- Depende fortemente de:
 - Renomeação de registradores
 - Agendamento dinâmico
 - Execução fora de ordem
- Múltiplas threads executam simultaneamente no mesmo ciclo
- Instruções de diferentes threads podem coexistir no pipeline

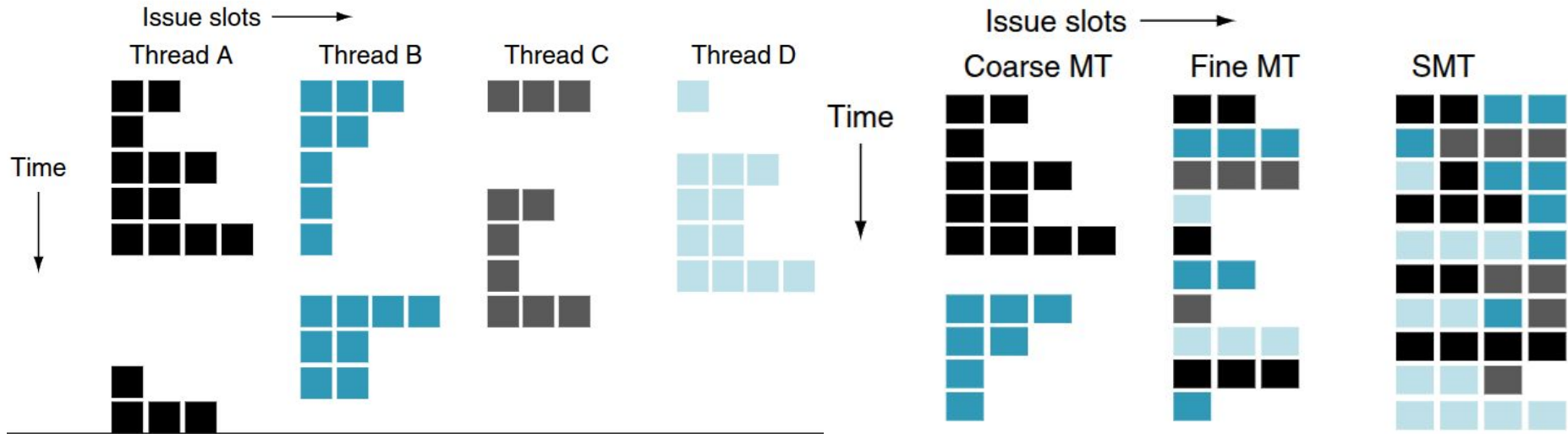
Hardware Multithreading

Limitações Práticas do SMT

- Recursos internos ainda são finitos
- Threads competem por:
 - Cache
 - ROB
 - Registradores físicos
 - Unidades funcionais
- Conflitos reduzem eficiência ideal
- Ganhos dependem da carga de trabalho

Hardware Multithreading

Ilustração conceitual das diferenças entre CGMT, FGMT e SMT



Hardware Multithreading

Caso Real: Intel Core i7

- Intel Core i7 suporta hardware multithreading
- Dois threads por núcleo
- Tecnologia comercialmente conhecida como *Hyper-Threading*
- Objetivo:
 - Melhor throughput
 - Melhor eficiência energética

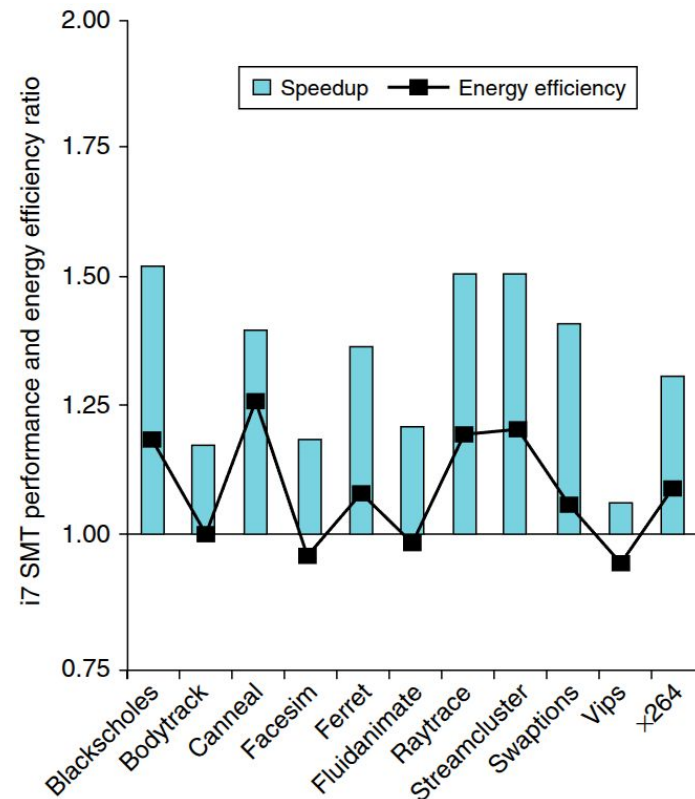
Hardware Multithreading

- **Resultados de Desempenho**

- Speed-up médio observado: $1.31\times$
- Demonstra alta relação custo-benefício
- Melhor aproveitamento das unidades funcionais

- **Resultados Energéticos**

- Eficiência energética média: $1.07\times$
- Melhor desempenho sem aumento proporcional de energia
- Multithreading melhora desempenho por watt



Hardware Multithreading

Síntese

- Hardware multithreading aumenta utilização do processador
- **Fine-grained:**
 - Excelente ocultação de stalls
 - Maior impacto na latência individual
- **Coarse-grained:**
 - Menor sobrecarga individual
 - Melhor para stalls longos
- **SMT:**
 - Explora simultaneamente TLP e ILP
 - Melhor utilização dos recursos do processador
 - Técnica amplamente utilizada em CPUs modernas

Multicore e outros multiprocessadores de memória compartilhada

Motivação para Arquiteturas Multicore

- O aumento da frequência dos processadores tornou-se limitado por consumo energético e dissipação térmica
- A evolução do desempenho passou a depender do paralelismo
- A Lei de Moore continuou aumentando o número de transistores disponíveis
- A solução adotada foi integrar múltiplos processadores no mesmo chip
- O desafio principal tornou-se a programação paralela eficiente

Multicore e outros multiprocessadores de memória compartilhada

O Problema da Programação Paralela

- Muitos programas antigos foram escritos para execução sequencial
- Reescrever aplicações para paralelismo é complexo e custoso
- Paralelismo introduz:
 - Sincronização
 - Compartilhamento de dados
 - Condições de corrida
 - Problemas de escalabilidade
- Arquiteturas modernas tentam simplificar esse modelo

Multicore e outros multiprocessadores de memória compartilhada

Estratégia para Simplificar o Paralelismo

- Uma solução foi fornecer um único espaço físico de endereçamento
- Todos os processadores enxergam a mesma memória
- Programadores não precisam controlar explicitamente onde os dados estão
- Variáveis podem ser acessadas por qualquer processador
- Comunicação ocorre naturalmente via memória compartilhada

Multicore e outros multiprocessadores de memória compartilhada

Espaço de Endereçamento Compartilhado

- Todos os processadores acessam o mesmo espaço físico de memória
- Qualquer núcleo pode realizar:
 - Loads
 - Stores
- Dados compartilhados tornam-se visíveis globalmente
- Comunicação entre threads/processadores ocorre via variáveis compartilhadas

Multicore e outros multiprocessadores de memória compartilhada

Alternativa: Espaço de Endereçamento Separado

- Outra abordagem usa memória privada para cada processador
- Compartilhamento precisa ser explícito
- Comunicação ocorre via troca de mensagens
- Programador precisa controlar movimentação dos dados
- Modelo comum em clusters distribuídos

Multicore e outros multiprocessadores de memória compartilhada

Shared Memory Multiprocessor (SMP)

- Todos os processadores compartilham o mesmo espaço físico de memória
- Comunicação ocorre através de variáveis compartilhadas
- Modelo dominante em processadores multicore modernos
- Tecnicamente, o nome mais preciso seria:
 - Shared-address multiprocessor

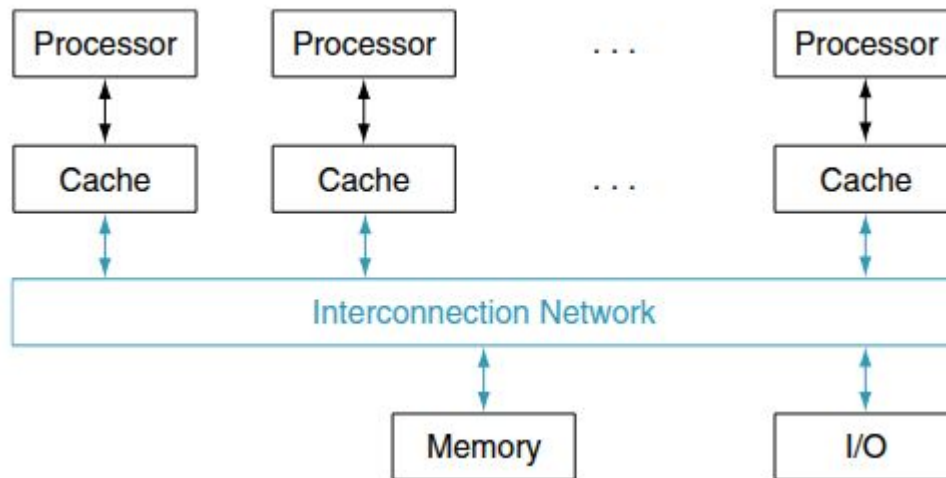
Multicore e outros multiprocessadores de memória compartilhada

Organização Clássica de um SMP

- Processadores conectam-se a uma memória compartilhada
- Cada núcleo possui caches privadas
- Sistema mantém visão consistente da memória
- Processadores podem executar aplicações independentes
- Espaços virtuais continuam separados por processo

Multicore e outros multiprocessadores de memória compartilhada

Organização clássica de um multiprocessador com memória compartilhada



Multicore e outros multiprocessadores de memória compartilhada

Memória Virtual em SMP

- Mesmo compartilhando memória física, processos podem possuir:
 - Espaços virtuais independentes
- Sistema operacional mantém isolamento entre aplicações
- Memória compartilhada pode coexistir com proteção de memória
- Virtualização continua sendo essencial

Multicore e outros multiprocessadores de memória compartilhada

O Problema da Coerência de Cache

- Cada processador possui cache própria
- Uma variável pode existir em múltiplas caches simultaneamente
- Atualizações em uma cache podem não aparecer imediatamente nas outras
- Isso gera inconsistência de dados
- Hardware implementa protocolos de coerência

Multicore e outros multiprocessadores de memória compartilhada

Principais modelos de SMP

- Existem dois principais modelos:
 1. **UMA (Uniform Memory Access)**
 2. **NUMA (Nonuniform Memory Access)**
- A diferença principal está na latência de acesso à memória
- Organização física da memória altera desempenho e escalabilidade

Multicore e outros multiprocessadores de memória compartilhada

Uniform Memory Access (UMA)

- Todos os processadores possuem mesma latência para acessar memória
- Tempo de acesso independe do processador solicitante
- Estrutura mais simples de programar
- Modelo típico de pequenos multiprocessadores
- Muito comum em multicore convencionais

Multicore e outros multiprocessadores de memória compartilhada

Características do Modelo UMA

- Memória é centralizada
- Interconexão normalmente uniforme
- Programação paralela é simplificada
- Escalabilidade limitada
- Contenção cresce com aumento de processadores

Multicore e outros multiprocessadores de memória compartilhada

Limitações do Modelo UMA

- Muitos núcleos competem pela mesma memória
- Barramentos/interconexões tornam-se gargalos
- Latência aumenta com escala
- Largura de banda compartilhada limita desempenho
- Escalabilidade é reduzida para grandes sistemas

Multicore e outros multiprocessadores de memória compartilhada

Nonuniform Memory Access (NUMA)

- Latência depende do processador e da localização dos dados
- Algumas regiões de memória são mais próximas de determinados núcleos
- Memória é fisicamente distribuída
- Processadores possuem acesso rápido à memória local
- Acesso remoto possui maior latência

Multicore e outros multiprocessadores de memória compartilhada

Funcionamento do NUMA

1. Cada processador possui memória local
2. Processador acessa memória local rapidamente
3. Memória remota exige comunicação pela interconexão
4. Latência varia conforme localização dos dados
5. Sistema operacional tenta otimizar afinidade de memória

Multicore e outros multiprocessadores de memória compartilhada

Vantagens do NUMA

- Melhor escalabilidade
- Redução da contenção de memória
- Maior largura de banda agregada
- Menor latência para memória local
- Permite sistemas com muitos processadores

Multicore e outros multiprocessadores de memória compartilhada

Desvantagens do NUMA

- Programação torna-se mais complexa
- Localidade de memória impacta fortemente desempenho
- Migração inadequada de threads reduz eficiência
- Balanceamento de memória é mais difícil

Multicore e outros multiprocessadores de memória compartilhada

Comparação entre UMA e NUMA

Característica	UMA	NUMA
Latência de memória	Uniforme	Variável
Escalabilidade	Menor	Maior
Complexidade de programação	Menor	Maior
Localidade de memória	Menos crítica	Muito crítica
Organização da memória	Centralizada	Distribuída

Multicore e outros multiprocessadores de memória compartilhada

Compartilhamento de Dados em Paralelo

- Processadores frequentemente compartilham estruturas de dados
- Operações paralelas podem acessar mesmas variáveis
- Sem coordenação adequada surgem inconsistências
- Necessário controlar acesso concorrente
- Surge a necessidade de sincronização

Multicore e outros multiprocessadores de memória compartilhada

O Que é Sincronização?

- Sincronização coordena acesso a dados compartilhados
- Garante ordem correta das operações
- Evita condições de corrida (race conditions)
- Controla regiões críticas do programa
- Fundamental em programação paralela

Multicore e outros multiprocessadores de memória compartilhada

Relação entre Multicore e Memória Compartilhada

- Multicore moderno normalmente utiliza SMP
- Núcleos compartilham memória física
- Coerência e sincronização tornam-se fundamentais
- Crescimento do número de núcleos aumenta complexidade
- NUMA surge como solução para escalabilidade

Exemplo de programa usando OpenMP

Um programa simples de processamento paralelo para um espaço de endereçamento compartilhado:

Suponha que queiramos somar 64.000 números em um computador multiprocessador de memória compartilhada com tempo de acesso à memória uniforme. Vamos supor que temos 64 processadores.

Exemplo de programa usando OpenMP

```
#define P 64 /* define a constant that we'll use a few times */  
#pragma omp parallel num_threads(P)  
  
#pragma omp parallel for  
for ( $Pn = 0$ ;  $Pn < P$ ;  $Pn += 1$ )  
    for ( $i = 1000 * Pn$ ;  $i < 1000 * (Pn + 1)$ ;  $i += 1$ )  
         $sum[Pn] += A[i]$ ; /*sum the assigned areas*/  
  
#pragma omp parallel for reduction(+ : FinalSum)  
for ( $i = 0$ ;  $i < P$ ;  $i += 1$ )  
     $FinalSum += sum[i]$ ; /* Reduce to a single number */
```

Bibliografia

Capítulo 6 - Computer Organization and Design RISC-V Edition: The Hardware Software Interface, David A. Patterson, John L. Hennessy. Editora Morgan Kaufmann, 2ª Edição, 2020.